

0.102. Une lettre qui tire — une cible B

Le 0.101, et une cible : un B en (16, 2). Touché, il disparaît, une note joue, le score monte, et un nouveau B apparaît au hasard, sur une case vide.

1. Ce qu'il faut comprendre

C'est le 0.101, avec une chose en plus : une cible. Un **B**, d'abord en (16, 2). Quand le tir arrive sur lui : **touché !** Le B disparaît, une note joue, le score monte d'un point, et un nouveau B apparaît **ailleurs**.

Deux variables pour la cible, `bx` et `by`, et une pour le **score**, écrit en bas avec `texte()` et `nombre()`.

Ce qui est nouveau ici (1) : un quatrième cas dans le pas du tir. Avant « l'arrivée n'est pas vide », on regarde : `nx == bx && ny == by`, « l'arrivée, c'est la cible ». Il faut le tester **avant** : le B n'est pas vide, et sinon le tir s'y écraserait comme sur un X, sans rien faire.

Touché : `vole = 0` (le tir disparaît), `effacer(bx, by, 1)` (le B aussi), `score = score + 1` et `nombre()` pour l'afficher, `note(1, D05, 10, 12)` (le 0.83), puis `placerB()`.

Ce qui est nouveau ici (2) : `placerB()`, une place au hasard, mais sur une case vide. `hasard() % 20` donne une colonne de 0 à 19, `hasard() % 17` une ligne de 0 à 16 (le 0.81.2). Mais la place tirée peut tomber sur un X, ou sur le A ! On recommence alors, avec `do { ... } while (...)` (le 0.7) : le bloc se fait **une fois**, puis **encore**, tant que `lire(bx, by) != 0`. À la sortie, la case est vide : on y pose le B.

Pourquoi `do ... while` et pas `while` ? Il faut tirer une place **avant** de pouvoir la regarder. `do` fait le bloc d'abord, et ne teste qu'après.

Déroulons : A en (2, 8). HAUT jusqu'en (2, 2), puis DROITE : le A va en (3, 2) et regarde à droite. Bouton A : départ en (4, 2), puis (5, 2)... (15, 2). Pas suivant : l'arrivée est (16, 2), c'est `bx`, `by`. Touché : le score passe à 1, et un B apparaît au hasard.

Essaie : donne 5 points par B (`score = score + 5`), ou change la note (`MI5`, `SOL5`).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1623 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	117 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
placerB()	55 o	← main() – ta fonction
main()	596 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase	10 o	← lire(), poser(), effacer(), texte() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
Reste	10 o	← placerB() – le reste d'une division
CopierTuiles	85 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
Hasard	25 o	← hasard() – tire un nombre au hasard
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	304 o	← CopierTuiles – 19 lettres sur 43
Notes	120 o	← note() – la table des notes de musique
"SCORE"	5 o	← texte() – un texte gravé
". "	1 o	← texte() – un texte gravé
". "	1 o	← texte() – un texte gravé

Lettres gravées : ABCEORSX0123456789.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert

Fonctions du langage appelées :

poser()	× 5	64 o sur place, dans placerB(), main() + AdresseCase (10 o)
note()	× 1	56 o sur place, dans main() + Notes (120 o)
texte()	× 3	47 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()	× 3	45 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
lire()	× 4	44 o sur place, dans placerB(), main() + AdresseCase (10 o)
bouton()	× 5	25 o sur place, dans main() + LireManette (37 o)
nombre()	× 2	20 o sur place, dans main() + EcrireNombre (33 o)
hasard()	× 2	6 o sur place, dans placerB() + Hasard (25 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	× 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

chaque_minuteur()	× 2	21 o sur place, dans main()
placerB()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : une cible B ----
// Le 0.101, et une cible : un B. Touché, il disparaît, un son joue,
// le score monte, et un nouveau B apparaît ailleurs, au hasard.
// ALPHABET[0] : le A   ALPHABET[1] : le B   ALPHABET[23] : le X

#include <hasard>      // tire un nombre au hasard
#include <lire>        // lit la tuile posée sur une case
#include <poser>       // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <texte>       // écrit un texte à l'écran
#include <nombre>      // écrit un nombre en chiffres
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
#include <effacer>     // efface des cases, ou tout le fond
#include <note>        // joue une note
#include <reste>       // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne
```

4. UNE VARIABLE — DX

```
uint8_t dx = 1;           // le sens où regarde le A
```

5. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

6. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

8. UNE VARIABLE — TX

```
uint8_t tx = 0;           // le tir : sa place...
```

9. UNE VARIABLE — TY

```
uint8_t ty = 0;
```

10. UNE VARIABLE — TDX

```
uint8_t tdx = 0;          // ...et son sens
```

11. UNE VARIABLE — TDY

```
uint8_t tdy = 0;
```

12. UNE VARIABLE — VOLE

```
uint8_t vole = 0;           // 1 = le tir est en vol ; 0 = pas de tir
```

13. UNE VARIABLE — BX

```
uint8_t bx = 16;           // NOUVEAU : la cible B : sa colonne...
```

14. UNE VARIABLE — BY

```
uint8_t by = 2;           // ...et sa ligne. La première est en (16, 2).
```

15. UNE VARIABLE — SCORE

```
uint8_t score = 0;         // NOUVEAU : les B touchés

// ---- NOUVEAU : un B à une place au hasard, sur une case vide ----
// hasard() % 20 : un nombre de 0 à 19 (le reste de la division par 20).
// do { ... } while (...) : on tire une place, et on RECOMMENCE tant qu'elle
// n'est pas vide (un X, le A, le tir...). Le bloc se fait au moins une fois.
```

16. UNE FONCTION — PLACERB()

```
void placerB() {
    do {
        bx = hasard() % 20;    // colonne 0 à 19
        by = hasard() % 17;    // ligne 0 à 16
    } while (lire(bx, by) != 0);
    poser(bx, by, ALPHABET[1]);
}
```

17. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t l = 4; l < 13; l++) {    // le mur de X (le 0.101)
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);
    poser(bx, by, ALPHABET[1]);           // NOUVEAU : la cible, à sa première place
    texte(0, 17, "SCORE");               // NOUVEAU : le score, tout en bas
    nombre(6, 17, score);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde (le 0.100) ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- A lance le tir (le 0.101) ----
        if (bouton(A) && vole == 0) {
            nx = ax + dx;
            ny = ay + dy;
            if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                vole = 1;
                tx = nx;
                ty = ny;
                tdx = dx;
                tdy = dy;
                texte(tx, ty, ".");
            }
        }

        // ---- le tir avance ----
        if (chaque(100)) {
            if (vole == 1) {
                nx = tx + tdx;
                ny = ty + tdy;
                effacer(tx, ty, 1);
                if (nx >= 20 || ny >= 17) {
```

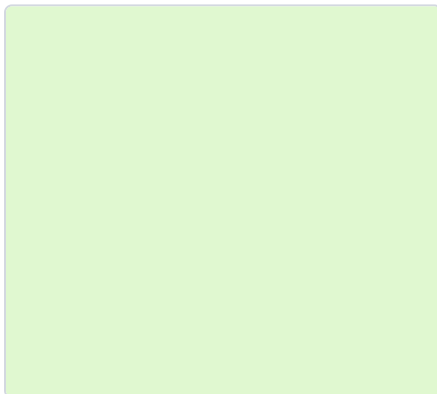
```

    vole = 0; // 1. hors de l'écran
} else if (nx == bx && ny == by) {
    // 2. NOUVEAU : l'arrivée, c'est la cible. TOUCHÉ !
    vole = 0; // le tir disparaît...
    effacer(bx, by, 1); // ...le B aussi...
    score = score + 1; // ...un point de plus...
    nombre(6, 17, score);
    note(1, D05, 10, 12); // ...un son (le 0.83)...
    placerB(); // ...et un nouveau B, ailleurs.
} else if (lire(nx, ny) != 0) {
    vole = 0; // 3. un X : il s'écrase
} else {
    tx = nx; // 4. une case vide : il y va
    ty = ny;
    texte(tx, ty, ".");
}
}
}
}
}

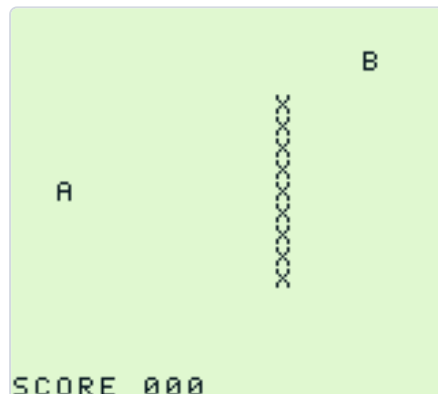
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



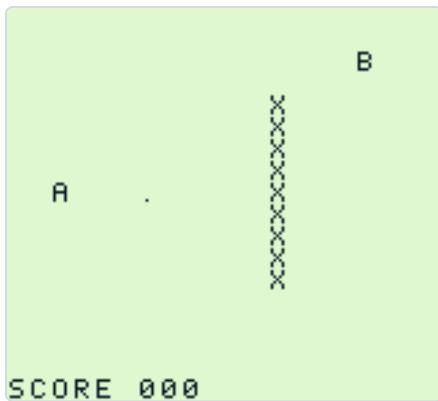
la toute première image — la console vient de démarrer — image 1



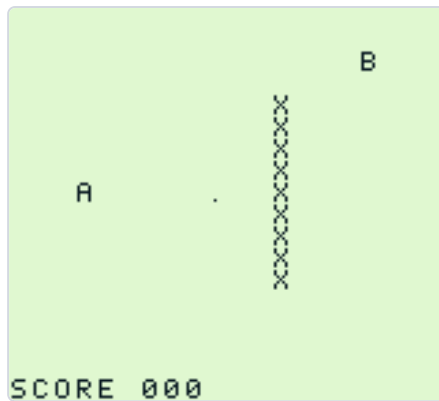
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

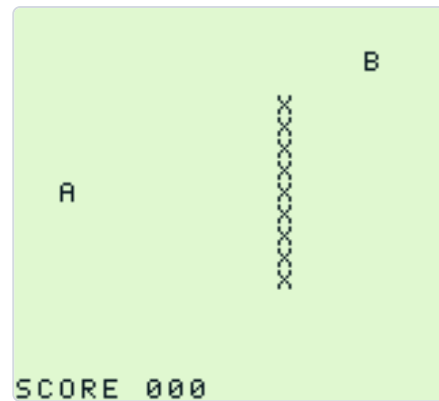
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



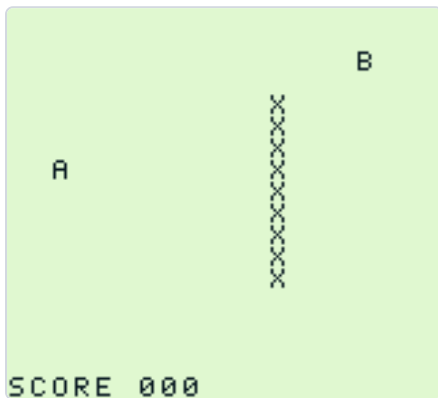
A — l'écran a changé



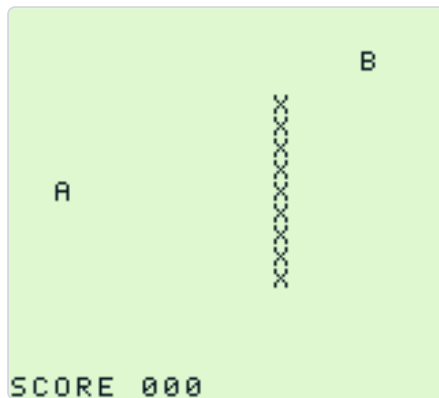
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Un B en (16, 2). Le tir qui l'atteint le fait disparaître avec une note ; le score passe à 1, et un nouveau B apparaît ailleurs, au hasard.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ : le B en (16, 2), SCORE 000
- ✓ le tir atteint le B : le score passe à 001
- ✓ le tir a disparu
- ✓ un seul B à l'écran : le nouveau, posé sur une case vide (en 12, 16)

Ensuite : **0.102.1. Une lettre qui tire — une cible B, et trois tirs rangés dans une struct** — chapitre 0 : Avant tout. · gameboy3
— écrire une cartouche Game Boy en C++